

AyudaVE

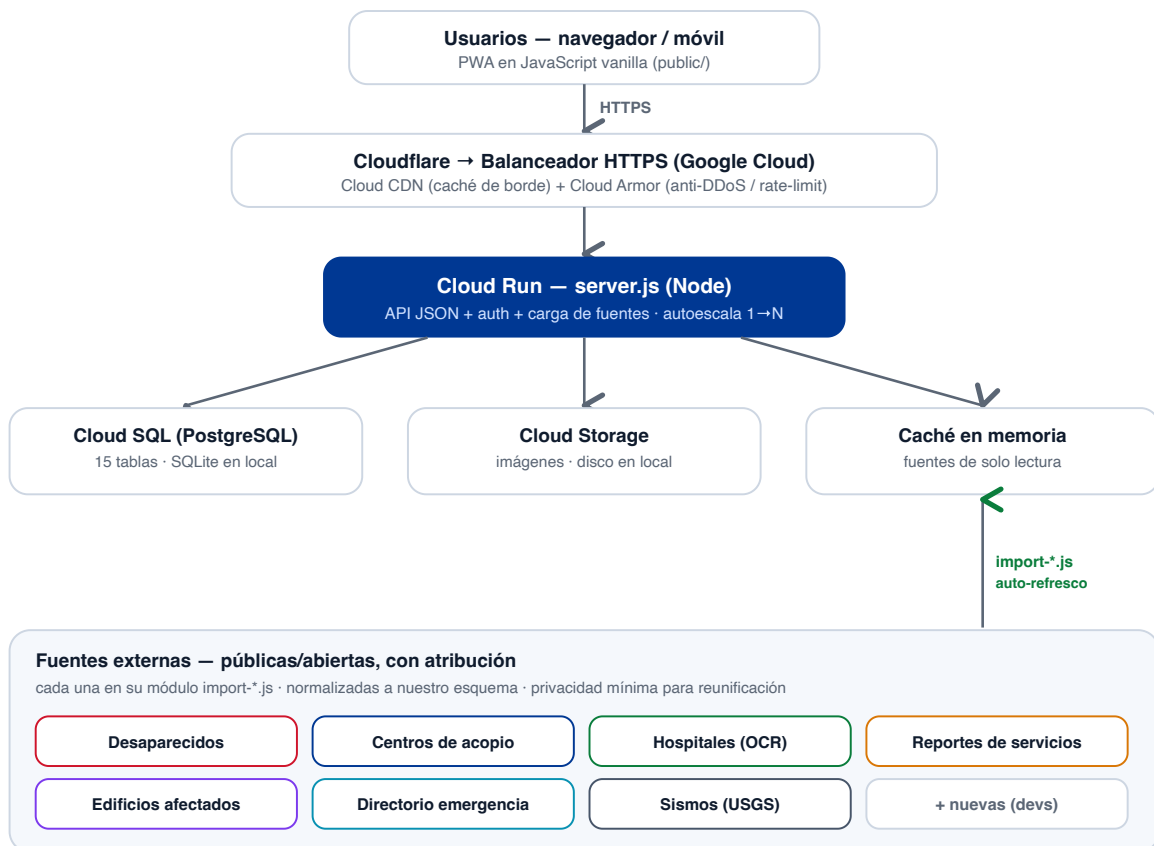
Documentación técnica

Versión 2.0 · Junio 2026

ayudahumanitariavenezuela.com · Documento técnico y operativo

AyudaVE — Arquitectura y flujo de datos

Node sin framework · base de datos y storage intercambiables por entorno · fuentes externas auto-sincronizadas



Contenido

- 1** Resumen ejecutivo
- 2** Funcionalidades
- 3** Arquitectura técnica
- 4** Fuentes de datos e integraciones
- 5** Modelo de datos
- 6** API / Endpoints
- 7** Seguridad y privacidad
- 8** Despliegue e infraestructura
- 9** Escalabilidad y alta disponibilidad
- 10** Operación (runbook)
- 11** Estructura del repositorio
- 12** Hoja de ruta y pendientes
- 13** Capturas de pantalla

Documento técnico y operativo de AyudaVE. **No contiene secretos** (contraseñas, claves de API, teléfonos de administradores, IDs de facturación): esos valores viven solo en las variables de entorno de producción y en notas privadas fuera del repositorio.

1. Resumen ejecutivo

AyudaVE es una aplicación web *mobile-first* de coordinación de ayuda humanitaria para Venezuela, creada para responder al **sismo de La Guaira (24–25 de junio de 2026)**, donde la rapidez y la accesibilidad son críticas. En vivo: ayudahumanitariavenezuela.com.

Objetivo	Principios de diseño
Reunir en un solo lugar, en tiempo real, la información dispersa que una familia o un voluntario necesita en la emergencia, y conectar a quien necesita ayuda con quien puede darla.	Sin fricción: buscar y donar no requieren registro; cuenta solo cuando es imprescindible. Resistente a picos de tráfico (no caerse en plena emergencia). No inventamos datos: agregamos fuentes abiertas con atribución. Privacidad primero: se publica el mínimo útil para reunificación. Identidad de Venezuela , neutral, sin colores de partidos.

Qué resuelve hoy: centros de acopio, personas desaparecidas, personas en hospitales, edificios afectados, cortes de servicios, directorio de emergencia y réplicas sísmicas — todo buscable y en un mapa, alimentado por **ocho fuentes externas** que se sincronizan solas.

Cobertura geográfica: la base de datos incluye los 24 estados de Venezuela con municipios y parroquias en selectores en cascada; el selector destaca los estados afectados por el sismo.

2. Funcionalidades

2.1 Buscar información (sin cuenta)

Módulo	Qué permite
Centros de acopio	Buscar/filtrar por estado, municipio o parroquia; tabla de conteos por zona; ver necesidades, dirección, cómo llegar y contacto directo (WhatsApp/Instagram). Los que tienen contacto aparecen primero.
Personas desaparecidas	Registro buscable (desaparecido/encontrado/fallecido) con auditoría de duplicados (se muestra un solo registro por persona).
Personas en hospitales	Registro buscable para reunificación familiar; búsqueda por nombre y por cédula (sin mostrar la cédula).
Edificios afectados	Daños estructurales con nivel (parcial/severo/total), estado, ubicación y "cómo llegar".
Mapa de la situación	Capas combinables: centros, cortes de servicios y edificios con daños.
Directorio de emergencia	Hospitales, bomberos, ambulancias y protección civil con teléfonos.
Recursos	Grupos de WhatsApp/Telegram, bases de datos y galerías curadas por el equipo.
Réplicas sísmicas	Sismos recientes (fuente USGS).

2.2 Dar ayuda

- **Donar:** insumos físicos, Pago Móvil, transferencia bancaria, cripto (USDT/USDC) o fondo general. Donación anónima disponible. Las donaciones son **privadas** (solo el dueño del centro o un admin las ve).
- **Ser voluntario:** registro con habilidades (carro, cargar, médico, etc.) y zona; acceso a tareas de centros que piden voluntarios.
- **Centro de acopio:** panel para gestionar necesidades, inventario, métodos de pago, horario, pedir voluntarios/transporte, registrar **movimientos** logísticos y cerrar el centro.
- **Pedir ayuda:** solicitudes de personas (rescate, insumos, etc.).
- **Reportar:** personas (desaparecida/fallecida/encontrada), avistamientos y mascotas.

2.3 Administración / verificación

Rol **Admin/Verificador** para verificar centros, moderar reportes, asignar admins a centros, ver el **panel de logística** y un **dashboard de métricas** (visitas, registros por hora/día, actividad). Se designa por la variable de entorno `AYUDAVE_ADMINS` (o `data/admins.json`).

3. Arquitectura técnica

Diseño deliberadamente **simple y portable**: sin framework de frontend, sin paso de build, y con componentes **intercambiables** que permiten correr igual en local (cero dependencias externas) que en producción gestionada.

Capa	Tecnología
Backend	Node.js (módulo <code>http</code> nativo, sin Express), CommonJS, Node ≥ 22.5 — todo en <code>server.js</code> .
Frontend	SPA *vanilla* JS sin build: router de pantallas en <code>app.js</code> , <code>styles.css</code> , <code>index.html</code> .
Persistencia	Intercambiable — <code>node:sqlite</code> en local / PostgreSQL (<code>pg</code>) si hay <code>DATABASE_URL</code> .
Imágenes	Intercambiable — disco local / Google Cloud Storage si hay <code>GCS_BUCKET</code> (servidas vía <code>/img/</code>).
Mapa	Leaflet + tiles CARTO (sin key) por defecto, o Google Maps (con key).
Datos geográficos	<code>public/geo-ve.js</code> — estados, municipios y parroquias.
HTTP	<code>undici</code> para las llamadas a fuentes externas.
Dependencias	Solo 3: <code>pg</code> , <code>@google-cloud/storage</code> , <code>undici</code> .

Patrón de intercambio — un único punto de decisión elige la implementación según el entorno, sin tocar el resto del código:

```
// store.js
module.exports = process.env.DATABASE_URL
  ? require('./store-pg')      // producción: PostgreSQL
  : require('./store-sqlite'); // local: node:sqlite

// storage.js → Google Cloud Storage si GCS_BUCKET, si no disco local
```

Ventaja: la misma base de código corre sin dependencias externas en una laptop y, con dos variables de entorno, en infraestructura gestionada escalable. Probado en SQLite y en PostgreSQL real.

El diagrama de arquitectura y flujo de datos está en `docs/arquitectura.svg` (y en la portada de este documento).

4. Fuentes de datos e integraciones

El corazón del proyecto: AyudaVE **agrega datos de otras iniciativas** (todas públicas/abiertas, con atribución) y los mantiene sincronizados. Cada fuente vive en su módulo `import-*.js`. Hay **dos patrones**:

- **En base de datos** (idempotente, reconciliable): se escribe a una tabla; para datos que se mezclan con lo creado por usuarios o requieren búsqueda en SQL a gran escala.
- **En memoria** (solo lectura, auto-refresco): caché en el proceso con degradación elegante (si la fuente cae, conserva la última copia buena); para datos que cambian seguido y no se editan localmente.

Módulo	Fuente	Qué trae	Destino	Refresco
<code>import-dtv.js</code> + <code>audit-dtv.js</code>	desaparecidosterremotovenezuela.com (API)	Personas desaparecidas/encontradas	tabla <code>persons</code>	Carga + refresco; auditoría de duplicados.
<code>import-acopio.js</code>	acopiovenezuela.vercel.app (Google Sheet vía sheet2api)	Centros de acopio	tabla <code>centers</code>	Auto ~1h (en el server).
<code>import-ocr-hospitals.js</code>	repo GitHub abierto (<code>ecrespo/OCR-...</code> , CSV)	Pacientes en hospitales	memoria	~6 h.
<code>import-reportes.js</code>	reporte-ve / ve.crafter.run (API)	Cortes de luz/agua/medicinas/comida...	memoria	~1h.
<code>import-edificios.js</code>	terremotovenezuela.com (API documentada)	Edificios afectados	memoria	~1h.
<code>import-directorio.js</code>	redayudavenezuela.com	Directorio de emergencia	tabla <code>directory</code>	Según fuente.
<code>import-sismos.js</code>	USGS (earthquake.usgs.gov)	Sismos/réplicas	memoria	Periódico.

Cómo agregar una fuente nueva (lo que harán los devs que se suman, en ~30 líneas): ver `docs/INTEGRACIONES.md`. Reglas: solo fuentes públicas/abiertas y **privacidad mínima** (no publicar cédula, dirección exacta, diagnósticos ni teléfonos privados; si un dato sensible sirve para buscar, hacerlo buscable **sin devolverlo**).

5. Modelo de datos

Una sola base (SQLite o Postgres según entorno), **15 tablas**. Patrón común: columnas "indexables" (id, estado, status...) + una columna `data` con el JSON completo del objeto.

Tabla	Columnas clave	Contenido
<code>centers</code>	id, name, status, estado, municipio, parroquia, distance, data	Centros de acopio (importados + creados por usuarios): necesidades, inventario, métodos de pago, horario, dueño (<code>ownerId</code>), verificación.
<code>donations</code>	id, center_id, center_name, estado, data	Donaciones a un centro o al fondo general (privadas).
<code>volunteers</code>	id, whatsapp, cedula, nombre, data	Voluntarios: perfil, habilidades, zona.
<code>applications</code>	id, volunteer_id, center_id, task, status	Postulaciones de voluntarios a tareas.
<code>persons</code>	id, status, nombre, estado, municipio, data	Personas reportadas (desaparecida/encontrada/fallecida).
<code>sightings</code>	id, person_id, data	Avistamientos / pistas sobre una persona.
<code>users</code>	id, phone (único), nombre, apellido, estado, municipio, parroquia, pin_hash, data	Cuentas (teléfono + PIN con scrypt), rol y perfil.
<code>sessions</code>	token (PK), user_id	Tokens de sesión (autenticación tipo Bearer).
<code>metrics</code>	k (PK), v	Contadores clave → valor (visitas, banderas de seed, marcas de refresco).
<code>visitor_days</code>	day, h	Visitantes únicos por día (hash anónimo, sin IP).
<code>resources</code>	id, type, title, url, descr, estado	Recursos compartidos por el equipo (grupos, bases, galería).
<code>movements</code>	id, center_id, type, data	Movimientos logísticos de un centro (entradas/salidas).
<code>directory</code>	id, category, name, zona, estado, data	Directorio de emergencia (hospitales/bomberos/ambulancias/protección civil).
<code>help_requests</code>	id, tipo, nombre, contacto, estado, municipio, urgencia, status, data	Solicitudes de ayuda de personas.
<code>pets</code>	id, status, tipo, zona, estado, data	Reportes de mascotas.

Las fuentes externas en **memoria** (hospitales OCR, reportes de servicios, edificios, sismos) **no** usan tabla: mantienen la BD liviana y evitan escribir datos de solo lectura que cambian seguido.

6. API / Endpoints

API REST servida bajo `/api/`. Las lecturas públicas (centros, personas, hospitales, edificios, reportes, métricas...) son cacheables por el CDN. Las acciones requieren sesión (token Bearer); los endpoints `/api/admin/*` requieren rol admin.

6.1 Centros y logística

Método	Ruta	Descripción
GET	<code>/api/centers</code>	Listar centros (público; oculta datos del dueño).
GET	<code>/api/centers/:id</code>	Detalle de un centro.
GET	<code>/api/centers/mine</code>	Centros del usuario autenticado.
POST	<code>/api/centers</code>	Crear centro (sesión).
PATCH	<code>/api/centers/:id</code>	Editar centro (dueño/admin).
POST	<code>/api/centers/:id/updates</code>	Publicar actualización en el hilo.
GET/POST	<code>/api/centers/:id/movements</code>	Movimientos logísticos del centro.
POST	<code>/api/movements/:id/estado</code>	Cambiar estado de un movimiento.

6.2 Donaciones, voluntarios, postulaciones

Método	Ruta	Descripción
GET	<code>/api/donations</code>	Listar donaciones (privado: dueño/admin).
POST	<code>/api/donations</code>	Registrar donación (anónima permitida).
PATCH	<code>/api/donations/:id</code>	Actualizar estado de donación.
POST	<code>/api/volunteers</code>	Registrar voluntario.
GET	<code>/api/volunteers/lookup</code>	Buscar perfil de voluntario.
PATCH	<code>/api/volunteers/:id</code>	Actualizar voluntario.
GET/POST	<code>/api/applications</code>	Listar / crear postulaciones.

6.3 Personas, avistamientos, mascotas, ayuda

Método	Ruta	Descripción
GET	/api/persons	Listar personas (público, cacheable, paginado).
GET	/api/persons/:id	Detalle de una persona.
GET	/api/persons/stats	Conteos por estado.
POST	/api/persons	Crear reporte de persona.
PATCH	/api/persons/:id	Actualizar reporte.
POST	/api/persons/:id/sightings	Registrar avistamiento.
GET/POST/PATCH	/api/pets · /api/pets/:id	Reportes de mascotas.
GET/POST/PATCH	/api/help-requests · /api/help-requests/:id	Solicitudes de ayuda.

6.4 Fuentes agregadas (solo lectura, cacheables)

Método	Ruta	Descripción
GET	/api/hospitals · /api/hospitals/summary	Personas en hospitales (búsqueda + resumen).
GET	/api/edificios	Edificios afectados.
GET	/api/reportes	Cortes de servicios.
GET	/api/sismos	Réplicas sísmicas.
GET	/api/directorio	Directorio de emergencia.
GET	/api/resources	Recursos compartidos.
GET	/api/audit	Resumen de auditoría de duplicados.

6.5 Cuentas, admin y utilidades

Método	Ruta	Descripción
GET	/api/users/check	¿Teléfono ya registrado?
POST	/api/users · /api/users/login	Registrar / iniciar sesión (token).
PATCH	/api/users/:id	Actualizar perfil.
GET	/api/admin/overview · /api/admin/dashboard · /api/admin/logistics	Paneles de admin.
POST	/api/admin/centers/:id · /api/admin/centers/:id/assign	Verificar/moderar/asignar centro.
POST	/api/admin/persons/:id	Moderar reporte.
POST	/api/resources · /api/admin/resources/:id/delete	Crear/borrar recurso (admin).
GET	/api/config	Configuración pública (p. ej. key de mapa).
POST	/api/upload	Subir imagen.
GET	/healthz	Chequeo de salud ({ "ok":true}).

Los endpoints de importación masiva (/api/admin/import-* , /api/admin/ingest) están **protegidos por token** y normalmente **desactivados**.

7. Seguridad y privacidad

Medida	Implementación
Autenticación	Teléfono +58 + PIN de 4 dígitos. PIN cifrado con scrypt + sal + pepper (irrecuperable). Sesión por token(<code>Authorization: Bearer</code>).
Normalización de teléfono	EIO inicial es indiferente: <code>0414...</code> , <code>414...</code> , <code>+58414...</code> = el mismo número.
Propiedad de centros	Cada centro tiene <code>ownerId</code> ; crear/editar/confirmar requiere ser dueño o admin.
Privacidad de donaciones	<code>GET /api/donations</code> no es público: solo dueño o admin.
Exposición mínima	Las lecturas públicas de centros ocultan <code>ownerId</code> / <code>ownerPhone</code> .
Contenido de usuarios	URLs validadas (solo http/https; se bloquea <code>javascript: / data: </code>); HTML escapado en texto libre.
Endurecimiento HTTP	CSP estricta, rate-limit por IP, bloqueo por intentos de login.
Integridad de assets	Cache-busting <code>?v=BUILD</code> : nunca se sirven versiones obsoletas tras un deploy.
Borde / red	Cloud Armor (WAF + rate-limit) y CDN delante del servicio.
Visitas anónimas	Contador con hash diario de IP+UA + sal; no se guarda la IP .

Privacidad de datos personales (postura del proyecto): de registros de personas se publica el **mínimo útil para encontrar a alguien** (nombre, hospital/zona, edad, estado). **No** se publican cédula, dirección/domicilio, diagnósticos médicos ni N° de historia. Cuando un dato sensible sirve para buscar (p. ej. la cédula), se hace **buscable del lado del servidor sin devolverlo**.

8. Despliegue e infraestructura

Arquitectura **serverless** y gestionada para escalar ante picos. Descrita por **tipo de servicio** (los identificadores, regiones, IPs y secretos reales no van en el repo):

Capa	Servicio	Rol
CDN / WAF	Cloudflare + Balanceador HTTPS de Google Cloud (+ Cloud Armor)	TLS, caché de borde, anti-DDoS, rate-limit.
Cómputo	Google Cloud Run	Contenedor Node; autoescala 1→N.
Base de datos	Cloud SQL (PostgreSQL)	Persistencia gestionada con respaldos.
Imágenes	Cloud Storage	Fotos subidas por usuarios.
Build/Deploy	Cloud Build (<code>gcloud run deploy --source .</code>)	Construye la imagen con el <code>Dockerfile</code> . No hay CI/CD : deploy manual.

Desplegar un cambio (renueva el cache-busting; nunca usar `--set-env-vars` sin la lista completa):

```
gcloud run deploy ayudave --source . --region=<region> --update-env-vars "BUILD=$(date +%s)"
```

Verificar (por el dominio público, no por el `*.run.app`):

```
curl -s -o /dev/null -w "%{http_code}\n" https://ayudahumanitariavenezuela.com/healthz # 200
```

Detalle de montaje inicial de infraestructura y variables de entorno: ver `docs/DESPLIEGUE.md` .

9. Escalabilidad y alta disponibilidad

AyudaVE es **lectura-intensiva** (mucha gente buscando, poca reportando), lo que se defiende muy bien con CDN + caché. **Regla de oro:** el tráfico nunca llega directo a un solo servidor; pasa por CDN → balanceador → instancias sin estado → base gestionada.

1. **CDN** absorbe el 90%+ del tráfico (estáticos + lecturas públicas cacheadas 10–30 s): un pico de búsquedas se sirve desde el borde y **no toca la base**.
2. **Cloud Run autoescala** 1→N; backend **sin estado** (no usa disco ni SQLite en prod).
3. **Postgres gestionado** con respaldos en vez de un archivo en disco.
4. **Imágenes en bucket** (no llenan ni dependen del disco del servidor).
5. **Cloud Armor** frena bots/DDoS; **alertas de presupuesto** evitan quedarse sin créditos.
6. Sin **punto único de falla**: si una instancia muere, el resto sigue.

Antes del próximo evento: prueba de carga (k6/Artillery) y confirmar `cache HIT` en el CDN. Subir `--max-instances` y el tier de Cloud SQL si hace falta.

10. Operación (runbook)

Situación	Acción
Desplegar	<code>gcloud run deploy ...</code> (cap. 8). Verificar <code>/healthz</code> y el nuevo <code>?v=BUILD</code> por el dominio.
Una fuente externa no actualiza	Probar su CLI: <code>node import-<fuente>.js</code> . Si la fuente cambió de formato, ajustar el <code>norm()</code> del módulo. La caché conserva la última copia buena, así que no se rompe la app.
Designar un admin	Añadir el teléfono a la variable <code>AYUDAVE_ADMINS</code> (no en código) y redespargar.
Pico de tráfico	Cloud Run autoescala solo; si se queda corto, subir <code>--max-instances</code> . El CDN absorbe lo demás.
Revisar duplicados de personas	Endpoint <code>/api/audit</code> y pantalla de auditoría (admin).
Métricas / difusión	Dashboard de admin (visitas por hora/día, registros, actividad).
Quedarse sin créditos	Alertas de presupuesto (50/90/100%).

11. Estructura del repositorio

```
server.js           Backend completo (http + rutas + auth + carga de fuentes)
store.js / store-sqlite.js / store-pg.js  Persistencia intercambiable
storage.js          Imágenes: disco o Cloud Storage
Dockerfile          Imagen para Cloud Run
import-*.js         Una fuente de datos externa por archivo (8 fuentes)
audit-dtv.js        Auditoría de duplicados de personas
refresh-dtv.js      Refresco incremental de personas (job/cron/CLI)

public/             Frontend (sin build)
  index.html app.js api.js data.js geo-ve.js styles.css vendor/leaflet/

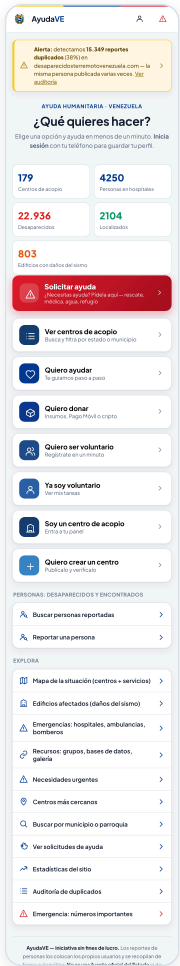
data/               Local: BD SQLite, uploads, notas privadas (gitignored)
docs/               Documentación (este set) + diagrama + generador de PDFs
```

12. Hoja de ruta y pendientes

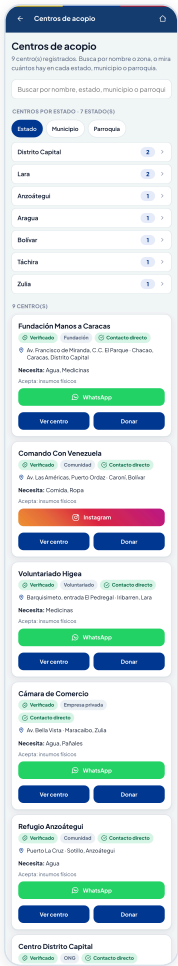
- **Más fuentes** integradas por los devs que se suman (refugios, agua potable, rutas seguras, etc.) — patrón en `INTEGRACIONES.md`.
- **Reportar a fuentes** que lo permitan (p. ej. la API de edificios acepta `POST` de reportes): contribuir de vuelta, no solo consumir.
- **Login por código** (WhatsApp/SMS) como alternativa al PIN.
- **Optimizaciones**: el escudo SVG pesa ~1.6 MB (optimizable); `npm audit` reporta vulnerabilidades moderadas en dependencias transitivas del SDK de Google Cloud Storage (riesgo bajo; se resuelven cuando Google actualice su SDK).

13. Capturas de pantalla

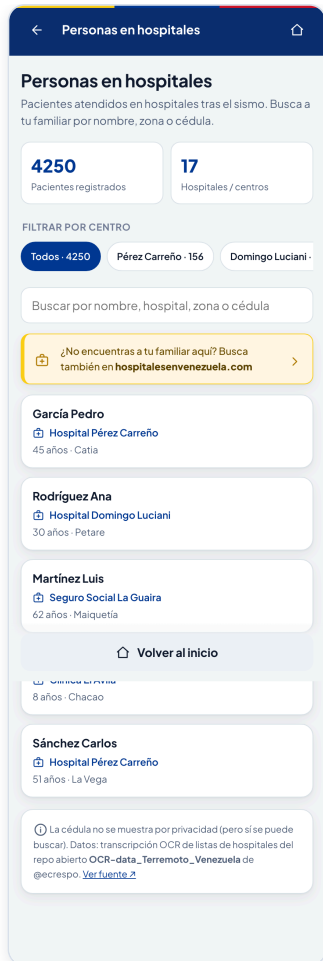
Pantallas principales de las funciones nuevas (con **datos de ejemplo**, sin información personal real). Las capturas se regeneran con `node docs/shoot-nuevas.js`.



Inicio — resumen en vivo (centros, hospitales, desaparecidos, edificios).



Centros de acopio — buscador + tabla de conteos por zona.



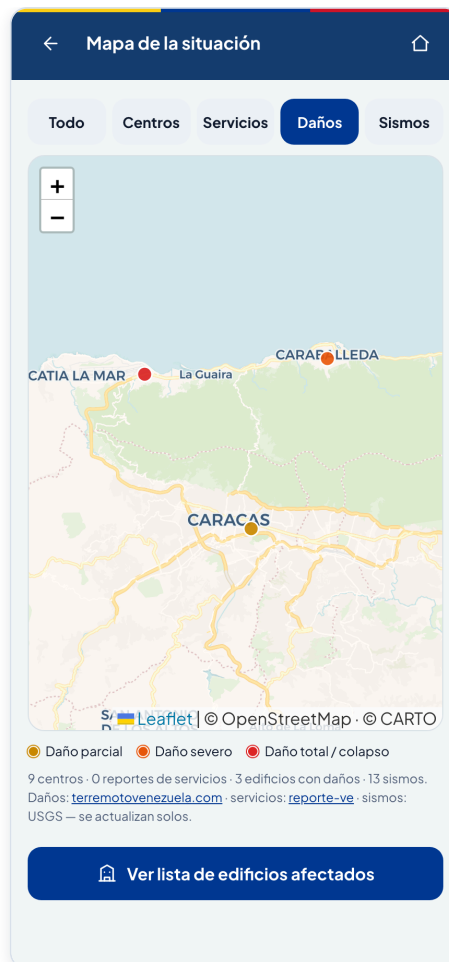
Personas en hospitales — búsqueda para reunificación familiar.



Edificios afectados — filtro por nivel de daño.



Recursos — grupos de WhatsApp/Telegram, bases de datos, galería.



Mapa de la situación — capas de centros, servicios y daños.